

SQL Server Unique Index

In this article, I am going to discuss **SQL Server Unique Index** with Examples. Please read our previous article where we discussed [How SQL Server Index impacts DML Operations](#). When we create an Index by using the Unique option then it is called Unique Index. Then the column(s) on which the unique index is created will not allow duplicate values i.e. it works as a unique constraint. The Unique Index in SQL Server gives guarantees that the column on which the index is created will not accept any duplicate values.

Example to Understand SQL Server Unique Index

We are going to use the following Employees table. Please use the following SQL Script to create and populate the Employees table with the required sample data.

```
CREATE TABLE Employees
(
    Id INT PRIMARY KEY,
    FastName VARCHAR(30),
    LastName VARCHAR(30),
    Salary INT,
    Gender VARCHAR(30),
    City VARCHAR(30)
)
```

As we marked the Id column as the Primary Key column in the Employees table, by default a **UNIQUE CLUSTERED INDEX** gets created in the Id column. To check whether a **UNIQUE CLUSTERED INDEX** is created or not use the **sp_helpindex** system stored procedure as shown below.

sp_helpindex Employees

index_name	index_description	index_ke
PK__Employee__3214EC0775FF9526	clustered, unique, primary key located on PRIMARY	Id

The above output clearly shows that a unique clustered index is created on the Id column of the Employees table. As we have a **UNIQUE CLUSTERED INDEX** on the Id column of the Employees table, any attempt to Insert duplicate key values i.e. the Id column values of the Employee table will throw an error. Let's try to insert two records with the same Id values as shown below.

```
INSERT INTO Employees VALUES(1,'Pranaya', 'Rout',4500,'Male','Mumbai')
INSERT INTO Employees VALUES(1,'Anurag', 'Mohanty',2500,'Male','Delhi')
```

When we try to insert the above two records, it gives us the below error which ensures that we cannot insert duplicate key values when there is a unique index in SQL Server.

'Violation of PRIMARY KEY constraint 'PK__Employee__3214EC0775FF9526'. Cannot insert duplicate key in object 'dbo.Employees'. The duplicate key value is (1)'.

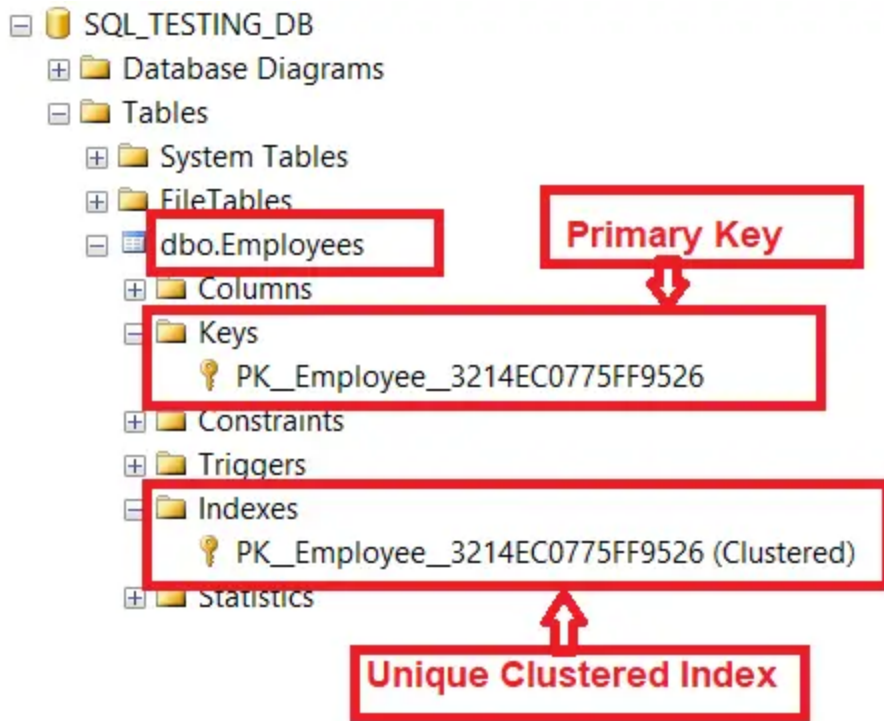
Now let's try to drop the Unique Clustered index on the Id column by using the DROP command as shown below.

DROP INDEX Employees.PK__Employee__3214EC0775FF9526

When we try to execute the above DROP INDEX command, it gives us the below error

'An explicit DROP INDEX is not allowed on index 'Employees.PK__Employee__3214EC0775FF9526'. It is being used for PRIMARY KEY constraint enforcement.'

So the above error message proves that the SQL server internally uses the **UNIQUE INDEX** to enforce the uniqueness of values and primary key. To see the primary key and Index, expand the keys folder in the object explorer window, and you can see the primary key constraint. Similarly, expand the indexes folder in the object explorer window and you can see the unique clustered index as shown in the below diagram.



You can see in the object explorer it just shows the clustered word only. To confirm whether it is a **UNIQUE** index or not, right-click on the index and select properties. The properties window shows the **UNIQUE** checkbox being selected as shown in the below diagram.

Index Properties - PK_Employee_3214EC0775FF9526

Ready

Select a page

- General
- Options
- Storage
- Fragmentation
- Extended Properties

Connection

LAPTOP-2HN3PT8T\SQL EXPRESS [sa]

[View connection properties](#)

Progress

Ready

Script Help

Table name: Employees

Index name: PK_Employee_3214EC0775FF9526

Index type: **Clustered** ☒ Unique

Index key columns

Name	Sort Order	Data Type	Size	Identity	Allow NULLs
Id	Ascending	int	4	No	No

Add... Remove Move Up

OK Cancel Help

DELETE Clustered Index in SQL Server

We cannot delete the Unique Clustered Index using the DROP Command but SQL Server allows us to delete the **UNIQUE CLUSTERED INDEX** from the object explorer. So Right-click on the index, and then select the **DELETE** option and finally click on the **OK** button. You will see that along with the UNIQUE index, the primary key constraint is also deleted.

Now, let's try to insert some duplicate values for the ID column of the Employees table and you will see that the rows get inserted into the table without any primary key violation error.

INSERT INTO Employees VALUES(2, 'Priyanka', 'Dwegaan', 4500, 'Female', 'Mumbai')
INSERT INTO Employees VALUES(2, 'Preety', 'Tiwary', 2500, 'Female', 'Delhi')

Once you insert the above two records, verify the Employee table as shown below.

SELECT * FROM Employees

Output:

Id	FastName	LastName	Salary	Gender	City
1	Pranaya	Rout	4500	Male	Mumbai
2	Priyanka	Dwegaan	4500	Female	Mumbai
2	Preety	Tiwary	2500	FEmale	Delhi

So this proves that the UNIQUE Index in SQL Server is used to enforce the uniqueness of values. The UNIQUENESS is a property of an Index in SQL Server and both CLUSTERED and NON-CLUSTERED

indexes can be created as UNIQUE.

We cannot create a unique index on a single column if that column contains NULL in more than one row. Similarly, we cannot create a unique index on multiple columns if the combination of columns contains NULL in more than one row. These are treated as duplicate values for indexing purposes. Let's create a Unique Non-Clustered Index on the FirstName and LastName columns of the Employees table

```
CREATE UNIQUE NONCLUSTERED INDEX UIX_Employees_FastNameee_LastName On  
Employees(FastName, LastName)
```

The above unique non-clustered index ensures that no 2 entries in the index have the same fast and last names. In the [Unique Constraint](#) article, we already discussed that a Unique Constraint in SQL Server can be used to enforce the uniqueness of values, across one or more columns.

Note: The Unique Constraint in SQL Server can be either unique clustered or unique non-clustered. While creating an index in SQL Server, if clustered or non-clustered is not specified by default it is non-clustered.

What are the differences between the UNIQUE Constraints and the UNIQUE Index?

There are no major differences between a unique constraint and a unique index in SQL Server. In fact, when we add a unique constraint, a unique index gets created behind the scenes. To understand this, let's add a unique constraint on the City column of the Employees table

```
ALTER TABLE Employees ADD CONSTRAINT UQ_Employees_City UNIQUE (City)
```

Now when we execute the above query it gives us the below error.

The CREATE UNIQUE INDEX statement terminated because a duplicate key was found for the object name 'dbo.Employees' and the index name 'UQ_Employees_City'. The duplicate key value is (Mumbai).

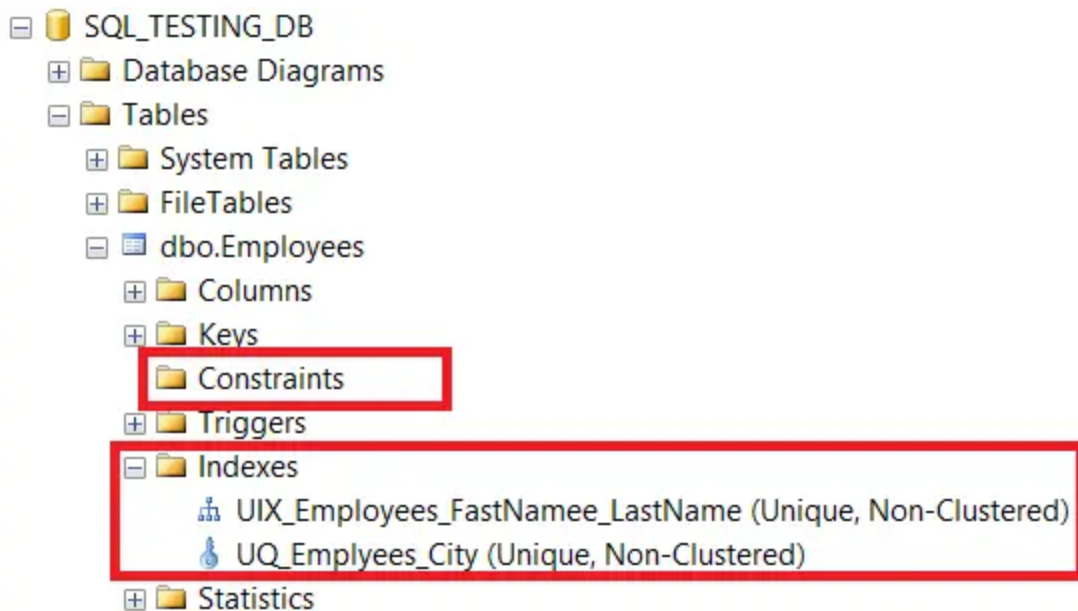
This is because we already have duplicate values on the City column of the Employees table and hence it doesn't allow creating Unique Index on the City Column. So let's truncate the table first and then create the Index as shown below.

```
TRUNCATE TABLE Employees
```

```
ALTER TABLE Employees ADD CONSTRAINT UQ_Employees_City UNIQUE (City)
```

Now, the statement is executed successfully.

At this point, you may expect a unique constraint to be created within the constraints folder. Refresh and expand the constraints folder in the object explorer window and you can see that the constraint is not present in this folder. Now, refresh and expand the 'indexes' folder and in the indexes folder, you can see a UNIQUE NON-CLUSTERED index with the name **UQ_Employees_City** as shown below.



You can also verify the available constraints of a table by using the system-stored procedure SP_HELPCONSTRAINT as shown below

EXECUTE SP_HELPCONSTRAINT Employees

It will give us the below result.

Object Name
Employees

constraint_type	constraint_name	delete_acti	update_act	status_enab	status_for_replica...	constraint_k
UNIQUE (non-clustered)	UQ_Employees_City	(n/a)	(n/a)	(n/a)	(n/a)	City

When we create a UNIQUE Constraint in SQL Server, it actually creates a UNIQUE index behind the scene. So, a UNIQUE index in SQL Server can be created explicitly by using the **CREATE INDEX statement** or indirectly by using a **UNIQUE constraint**.

When should you be creating a unique constraint over a unique index?

To make the intentions clear create a unique constraint when data integrity is the objective. This makes the objective of the index very clear. In either case, the data is validated in the same manner, and the query optimizer does not differentiate between a unique index created by a unique constraint or manually created.

Points to Remember:

When we create a PRIMARY KEY constraint in SQL Server, a unique clustered index on the column or columns is automatically created if a clustered index on the table does not already exist.

When we create a UNIQUE constraint in SQL Server, a unique non-clustered index is created automatically to enforce a UNIQUE constraint by default.

We cannot create a UNIQUE constraint or a UNIQUE index in SQL Server on an existing table if the table contains duplicate values in the key columns. To solve this, you need to remove the key columns from the

index definition or delete or update the duplicate values.

By default, duplicate values are not allowed on key columns, when you have a unique index or constraint. For, example if you try to insert 10 rows, out of which 5 rows contain duplicates, then all 10 rows are rejected. However, if I want only the 5 duplicate rows to be rejected and accept the non-duplicate 5 rows, then I can use the IGNORE_DUP_KEY option. An example of using the IGNORE_DUP_KEY option is shown below.

CREATE UNIQUE INDEX IX_Employees_City ON Employees(City) WITH IGNORE_DUP_KEY

Limitations and Restrictions

1. The unique index, UNIQUE constraint, or PRIMARY KEY constraint cannot be created if duplicate key values exist in the data.
2. A unique non-clustered index can contain included non-key columns

In the next article, I will discuss how to use [Index in GROUP BY Clause in SQL Server](#) with examples. Here, in this article, I try to explain the **SQL Server Unique Index** with Examples. I hope this SQL Server Unique Index with Examples article will help you with your need. I would like to have your feedback. Please post your feedback, question, or comments about this SQL Server Unique Index with Examples article.